

APIVizer v4.0.0 PHANTOM

Technical Documentation

Autonomous API Security Exploit Framework

Version: 4.0.0 PHANTOM | 36,198 Lines | 40 Modules
Last Updated: May 29, 2026

Metric	Value
Total Lines of Code	36,198
Exploit Engines	9 Autonomous Engines
Exploit Scenarios	88+ Elite Payloads
Upgrade Modules	40
Vulnerability Categories	12 MITRE ATT&CK Mapped
CI/CD Integrations	4 (GitHub/GitLab/Jenkins/Azure)
Export Formats	HTML, PDF, SARIF, DefectDojo, Nuclei
Plugin System	Hot-loadable .py modules

1. Architecture Overview

APIVizer PHANTOM is a single-file Python autonomous API security scanner that processes Postman collections and performs real exploit testing against live endpoints.

Core Components:

- **Collection Parser** - Parses Postman v2.1 collections with variable resolution
- **Elite Exploit Agent** - 88+ exploit scenarios with behavioral analysis + dynamic payloads
- **9 Autonomous Engines** - BOLA, JWT, SSRF, Race Condition, GraphQL, Mass Assignment, WAF Bypass, Credential Enum, Shadow API
- **40 Upgrade Modules** - Enterprise features (see Section 4)
- **Report Generator** - HTML, PDF, SARIF, Nuclei, DefectDojo exports
- **Plugin System** - Hot-load custom exploit modules
- **MCP Server Mode** - Cursor/Windsurf IDE integration via Model Context Protocol

Execution Flow:

1. Load Postman collection + environment variables
2. Auto-reconnaissance: fingerprint tech stack, detect gateway, discover hidden endpoints
3. For each request: Elite Exploit Agent tests all 88+ scenarios with dynamic payloads
4. Autonomous engines run specialized attacks (JWT cracking, BOLA probing, etc.)
5. Kill Chain Automator chains findings into multi-step attack paths
6. Strict confirmation: only report exploits with hard proof
7. Generate reports in multiple formats with remediation guidance

2. Autonomous Exploit Engines

BOLA/IDOR Engine (`_BOLAExploitEngine`)

Tests object-level authorization by manipulating IDs. Confirms only when different user data is returned.

JWT Attack Engine (`_JWTExploitEngine`)

alg:none, key confusion (RS256 to HS256), 80+ secret brute-force, kid injection, jku/x5u spoofing, claim tampering.

SSRF Callback Server (`_SSRFCallbackServer`)

Launches own HTTP callback server. Tests internal network, cloud metadata (AWS/GCP/Azure), DNS rebinding.

Race Condition Engine (`_RaceConditionEngine`)

Sends 10-50 concurrent requests to detect TOCTOU bugs. Confirms via duplicate creation or balance issues.

GraphQL Exploit Engine (`_GraphQLExploitEngine`)

Introspection dump, batch query abuse, field suggestion, nested query DoS, mutation tampering.

Mass Assignment Engine (`_MassAssignmentEngine`)

Injects admin/role/privilege fields. Tests hidden parameter acceptance on create/update endpoints.

WAF Bypass Engine (`_WAFBypassEngine`)

Multi-encoding payloads (Unicode, hex, double-URL), header smuggling, chunked encoding bypass.

Credential Enum Engine (`_CredentialEnumEngine`)

Timing-based username enumeration, default credentials, account lockout detection.

Shadow API Engine (`_ShadowAPIEngine`)

Discovers undocumented endpoints via wordlist brute-force, method probing, version guessing.

3. Elite Exploit Agent

The `_EliteExploitAgent` tests every request parameter through 4 phases:

- **Phase 1 - Static Payloads:** 88+ categories (SQLi, XSS, SSTI, LFI, RFI, CMD, LDAP, XPath, NoSQL, CRLF, Deserialization)
- **Phase 2 - Behavioral Learning:** Baseline request, analyze response patterns (status, timing, length, errors)
- **Phase 3 - Dynamic Payload Generation:** Targeted payloads based on learned behavior
- **Phase 4 - Strict Confirmation:** Only reports with hard proof (DB errors, reflected payloads, timing delays)

Confirmation Criteria (Anti-Spam):

- SQL Injection: Must see DB error keywords (mysql_fetch, ORA-, pg_, sqlite3)
- XSS: Payload must appear reflected in response body unchanged
- Command Injection: Must see OS output (root:x:, uid=, Windows\system32)
- SSTI: Must see computed result (49 from 7*7)
- Timing: Response delay must exceed 5 seconds vs baseline
- SSRF: Callback server must receive hit OR cloud metadata returned
- Auth Bypass: Must receive 200 with actual data, not generic error page

4. Upgrade Modules (1-40)

All modules are additive. No existing functionality is removed.

Modules 1-20:

#	Module	Class	Description
1	Auto-Remediation	_RemediationEngine	Fix code generation (Python/Node/Java)
2	Scan Diff	_ScanDiffEngine	Compare scans, track new/fixed vulns
3	Webhooks	_WebhookNotifier	Slack/Teams alerts on completion
4	PDF Export	_PDFReportGenerator	Executive-ready PDF reports
5	Exclusions	_ExclusionManager	.apivizer.yaml config
6	Auth Refresh	_AuthRefreshManager	Auto-refresh tokens mid-scan
7	Dependency Map	_EndpointDependencyMapper	Request chain mapping
8	Concurrent Scan	_ConcurrentScanner	Multi-threaded scanning
9	Checkpoint	_ScanCheckpointManager	Save/resume scans
10	Rate Throttle	_RateLimitHandler	Auto-adjust on 429s
11	Progress Bar	_ProgressTracker	Real-time progress + ETA
12	Exec Summary	_ExecutiveSummaryGenerator	Email-ready summary
13	Estimator	_ScanEstimator	Pre-scan time estimation
14	Recon	_ReconEngine	Tech stack fingerprinting
15	Discovery	_EndpointDiscovery	90+ hidden path brute-force
16	Schema Extract	_SchemaExtractor	OpenAPI/Swagger extraction
17	Cred Harvest	_CredentialHarvester	28+ secret pattern detection
18	Kill Chain	_KillChainAutomator	Chain exploits automatically
19	Compliance	_ComplianceReportGenerator	PCI/SOC2/HIPAA/ISO mapping
20	Training	_TrainingRecommender	Dev training recommendations

Modules 21-40:

#	Module	Class	Description
21	Attack Score	_AttackSurfaceScorer	0-100 security posture
22	Dashboard	_DashboardExporter	Grafana/Kibana export
23	Interactive CLI	_InteractiveCLI	Menu-driven interface
24	Update Check	_UpdateChecker	Version checking
25	Validator	_CollectionValidator	Pre-scan validation
26	WebSocket	_WebSocketExploitEngine	WS injection, CSWSH, tampering
27	gRPC Fuzzer	_GRPCFuzzer	Protobuf fuzzing, reflection probe
28	OAuth2/OIDC	_OAuth2ExploitEngine	PKCE bypass, token exchange, secret brute
29	Business Logic	_BusinessLogicExploitEngine	Price/qty manipulation, workflow bypass
30	API Versioning	_APIVersionScanner	Old version discovery, downgrade
31	Rate Bypass	_RateLimitBypassEngine	14 IP-spoof headers, mutations
32	BFLA	_BFLAExploitEngine	Admin endpoint probing, priv-esc
33	Nuclei Gen	_NucleiTemplateGenerator	YAML template generation
34	SARIF Export	_SARIFExporter	SARIF 2.1.0 + DefectDojo
35	CI/CD	_CICDIntegration	GitHub/GitLab/Jenkins/Azure
36	Plugins	_PluginSystem	Hot-load .py exploit modules
37	Multi-Collection	_MultiCollectionWorkflow	Chain collections with deps
38	Gateway Bypass	_APIGatewayBypassEngine	Detect 8 gateways + bypass
39	Fingerprint	_ResponseFingerprintEngine	Framework/DB/WAF detection
40	Scan Compare	_ScanComparison	Diff scans, trend analysis

5. Export Formats & Integration

Output Formats:

- **HTML Report** - Interactive with severity charts, finding details, proof
- **PDF Report** - Executive-ready with remediation (Module 4)
- **SARIF 2.1.0** - GitHub Security tab, CodeQL, DefectDojo (Module 34)
- **Nuclei Templates** - Auto-generated YAML for ProjectDiscovery (Module 33)
- **DefectDojo JSON** - Direct import format (Module 34)
- **JSON Checkpoint** - Resume interrupted scans (Module 9)
- **Grafana/Kibana** - Time-series dashboard data (Module 22)
- **Scan Comparison** - Diff report new/fixed/persistent (Module 40)

CI/CD Integration (Module 35):

- GitHub Actions - SARIF upload to Security tab
- GitLab CI - SAST report integration
- Jenkins - Jenkinsfile with email notifications
- Azure DevOps - Pipeline YAML

Plugin System (Module 36):

Drop .py files into plugins/ directory. Each plugin implements PLUGIN_NAME, PLUGIN_VERSION, PLUGIN_AUTHOR and a run() function.

6. Security & Dependencies

Dependency CVE Status:

Package	Min Version	CVE Status
Python	3.8+	No CVEs
requests	2.28+	No CVEs
urllib3	2.0+	No CVEs
reportlab (optional)	4.0+	No CVEs
flask-cors (optional)	6.0+	4 MEDIUM (fixed in 6.0)

Code Security Audit:

- ✓ No hardcoded credentials in source
- ✓ No eval()/exec() on user input
- ✓ No subprocess shell=True
- ✓ verify=False intentional (scanner must test self-signed targets)
- ✓ All payloads sent TO targets, never executed locally
- ✓ No SQL injection in tool code
- ✓ No insecure random usage

7. MITRE ATT&CK; Mapping

Vulnerability	MITRE ID	Tactic
SQL Injection	T1190	Initial Access
XSS	T1189	Initial Access
SSRF	T1090	Command & Control
JWT Attacks	T1550.001	Lateral Movement
BOLA/IDOR	T1078	Privilege Escalation
Mass Assignment	T1098	Persistence
Race Condition	T1499	Impact
Command Injection	T1059	Execution
Path Traversal	T1083	Discovery
SSTI	T1059.006	Execution
Broken Auth	T1110	Credential Access
Business Logic	T1565	Impact
OAuth Abuse	T1550	Lateral Movement
API Gateway Bypass	T1090.002	Command & Control

8. Usage & Command Reference

Basic Scan:

```
python apivizer.py
```

With Arguments:

```
python apivizer.py --collection api.postman_collection.json
```

```
python apivizer.py --collection api.json --environment env.json
```

```
python apivizer.py --collection api.json --output sarif
```

```
python apivizer.py --collection api.json --fail-on high
```

```
python apivizer.py --collection api.json --auth-header "Bearer TOKEN"
```

MCP Server Mode:

```
python apivizer.py --mcp
```

Exposes scan functions as MCP tools for Cursor, Windsurf, and other AI IDEs.

9. File Structure

apivizer.py (36,198 lines) - Single-file deployment. Optional deps: reportlab (PDF), flask-cors (MCP).

- Lines 1-500: Imports, constants, knowledge base, MITRE mappings
- Lines 500-5000: Core scanner, collection parser, request executor
- Lines 5000-15000: 9 Autonomous exploit engines
- Lines 15000-22000: Elite Exploit Agent (88+ scenarios)
- Lines 22000-30000: Upgrade Modules 1-25
- Lines 30000-36000: Upgrade Modules 26-40
- Lines 36000-36198: Main entry point, CLI argument parsing